

# Extra 2 · Transactions, ACID and Isolation Levels

**Why this matters in interviews.** "What is ACID?", "What are dirty, non-repeatable and phantom reads?", "What isolation levels exist?", "What is a deadlock and how do you avoid it?" — one of the most frequent database theory questions, asked of everyone from freshers to senior engineers. Chapter 05 taught `COMMIT` and `ROLLBACK`; this chapter explains *why* they exist and what happens when many users work at the same time.

**In this chapter:** what a transaction is · ACID · concurrency problems (lost update, dirty read, non-repeatable read, phantom) · isolation levels · how Oracle differs · locks and `SELECT ... FOR UPDATE` · deadlocks · optimistic vs pessimistic locking · savepoints.

## E2.1 What is a transaction?

A **transaction** is a group of statements that must be treated as **one indivisible unit of work**: either **all of it happens or none of it does**.

The textbook example is a bank transfer of ₹300 from Asha to Bala. It needs two updates, and it would be a disaster if only one of them happened.

**accounts**

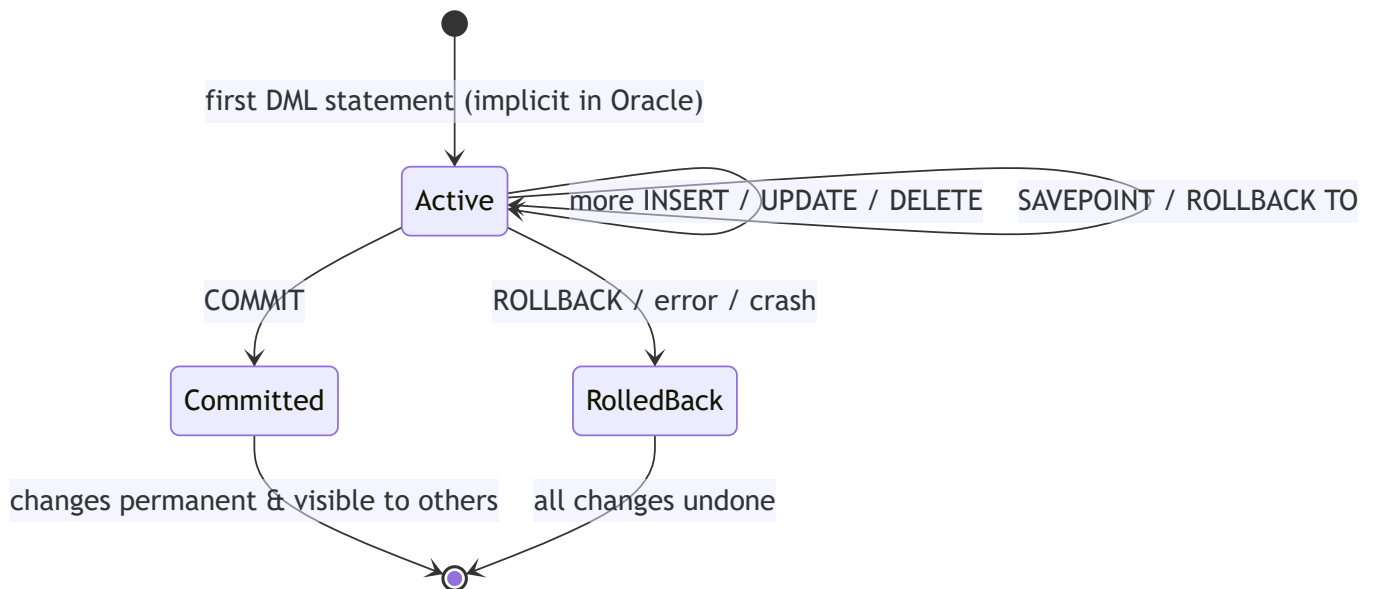
| id | owner | balance |
|----|-------|---------|
| 1  | Asha  | 1000    |
| 2  | Bala  | 500     |

```
UPDATE accounts SET balance = balance - 300 WHERE owner = 'Asha';
UPDATE accounts SET balance = balance + 300 WHERE owner = 'Bala';
COMMIT;
```

**Table after**

| id | owner | balance |
|----|-------|---------|
| 1  | Asha  | 700     |
| 2  | Bala  | 800     |

Both changes became permanent together. If the power failed *between* the two `UPDATE` s, the database must undo the first one on restart; money must never vanish.

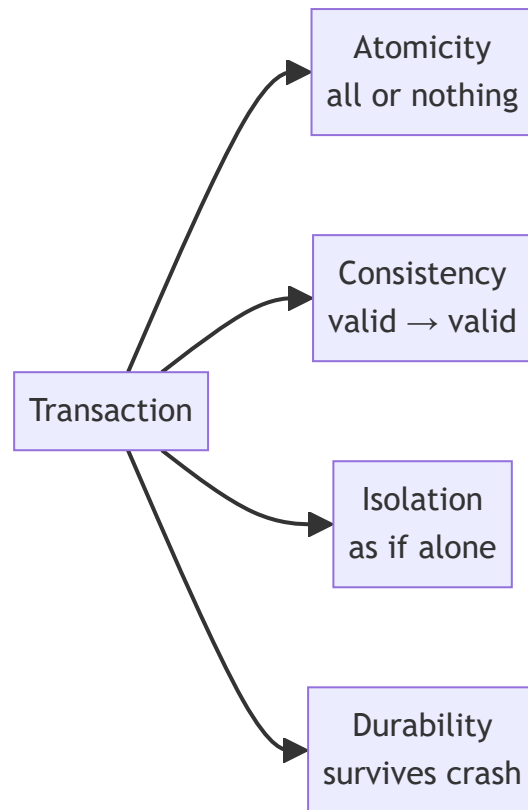


**Oracle vs others.** In Oracle a transaction **starts automatically** with your first DML statement and lasts until `COMMIT`, `ROLLBACK`, a DDL statement (which commits implicitly) or the session ending. SQL Server and MySQL in their default *autocommit* mode commit every single statement unless you say `BEGIN TRANSACTION` / `START TRANSACTION`. PostgreSQL is the same — *autocommit* unless `BEGIN`.

## E2.2 ACID

The four guarantees a transactional database gives.

| Letter   | Property           | Meaning                                                                                   | Example                                                                       | Enforced by                               |
|----------|--------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------|
| <b>A</b> | <b>Atomicity</b>   | all statements succeed, or none do                                                        | the two <code>UPDATE</code> s of the transfer                                 | undo / rollback data                      |
| <b>C</b> | <b>Consistency</b> | a transaction takes the database from one valid state to another — every constraint holds | total money is unchanged; no <code>CHECK</code> / <code>FK</code> is violated | constraints, triggers + application logic |
| <b>I</b> | <b>Isolation</b>   | concurrent transactions don't see each other's half-finished work                         | another user never sees ₹300 missing from Asha but not yet in Bala            | locking / multi-versioning                |
| <b>D</b> | <b>Durability</b>  | once committed, it survives crashes and power loss                                        | after <code>COMMIT</code> returns, a restart still has the data               | redo / write-ahead log                    |



**How they are implemented (short version).** Before a change is applied, the old value is kept in **undo** storage (this is what `ROLLBACK` reads, and also what gives other sessions a consistent view). Every change is also written to the **redo log** before `COMMIT` returns, so a crash can be replayed. This "write-ahead log" idea is common to all databases.

**Trap.** "Consistency" in ACID is **not** the same word as "consistency" in the CAP theorem or "eventual consistency". ACID-C means *the rules of the database are never broken*.

## Atomicity in action — rollback

```
UPDATE accounts SET balance = balance - 300 WHERE owner = 'Asha';  
-- the second update fails (e.g. wrong account); we abandon the whole transfer  
ROLLBACK;
```

### Table after

| id | owner | balance |
|----|-------|---------|
| 1  | Asha  | 1000    |
| 2  | Bala  | 500     |

Asha's balance is back to 1000 — the first statement was undone because the unit of work did not finish.

## Savepoints — undo part of a transaction

```
UPDATE accounts SET balance = balance - 100 WHERE owner = 'Asha';
SAVEPOINT after_first;
UPDATE accounts SET balance = balance - 999 WHERE owner = 'Asha'; -- a mistake
ROLLBACK TO after_first; -- undo only the mistake
COMMIT;
```

### Table after

| id | owner | balance |
|----|-------|---------|
| 1  | Asha  | 900     |
| 2  | Bala  | 500     |

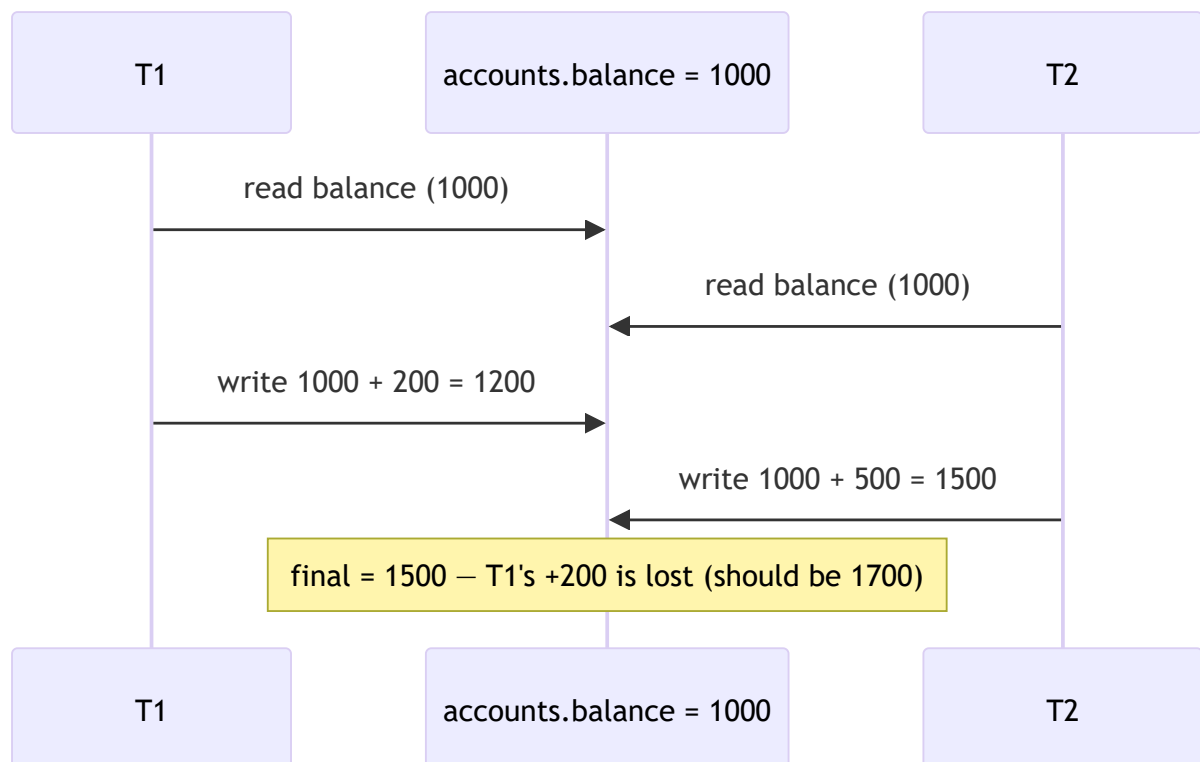
`ROLLBACK TO savepoint` undoes only what came after the savepoint; the transaction stays open and the first update is committed.

## E2.3 What can go wrong when users run at the same time

Without isolation, two transactions running together can interfere. There are four classic problems. In each diagram, time flows downward.

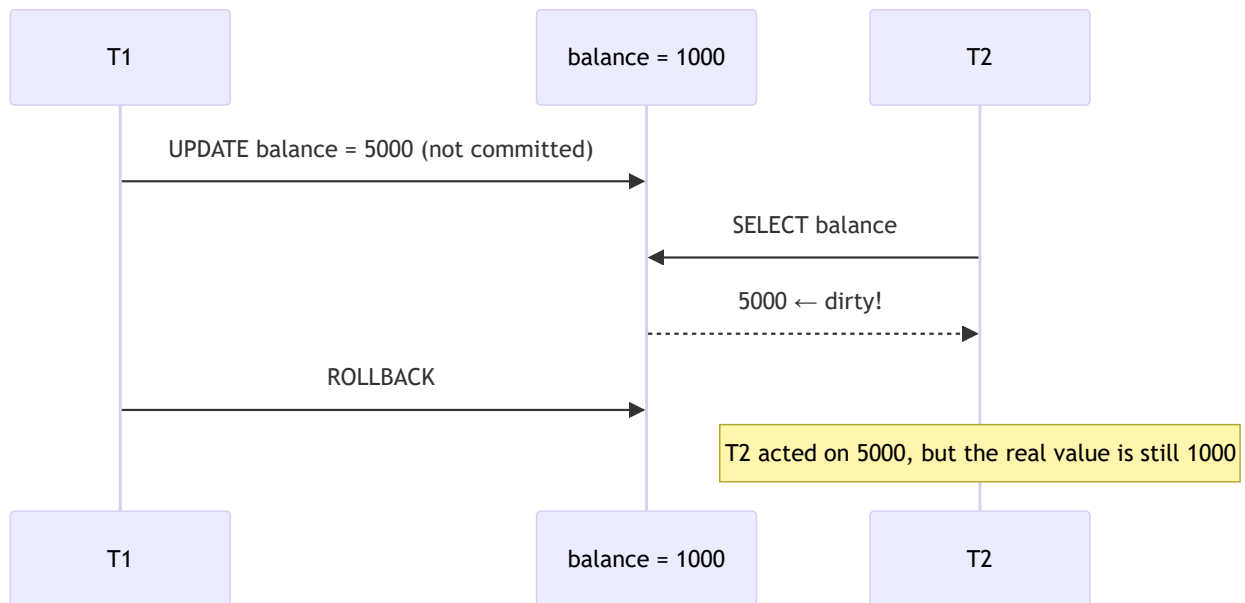
### 1. Lost update

Two transactions read the same value, both change it, and the later write **overwrites** the earlier one.



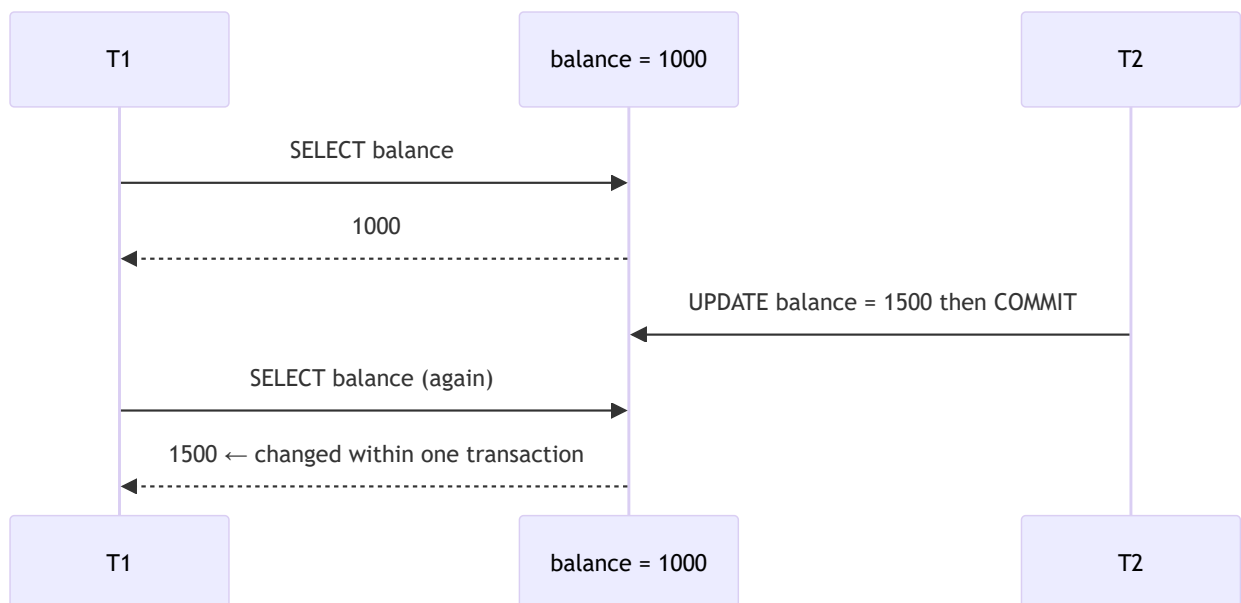
## 2. Dirty read

T2 reads data that T1 has changed but **not yet committed**. If T1 then rolls back, T2 has seen a value that never officially existed.



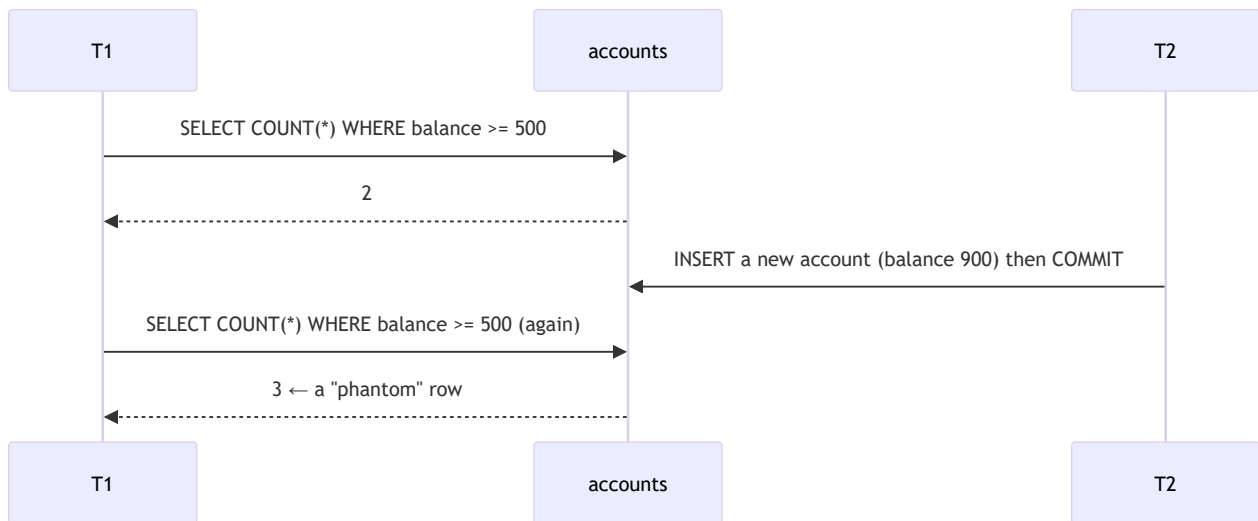
## 3. Non-repeatable read

T1 reads the **same row twice** and gets different values because T2 updated and committed it in between.



## 4. Phantom read

T1 runs the same **query** twice and the second time extra **rows** appear (or disappear) because T2 inserted/deleted rows that match the condition.



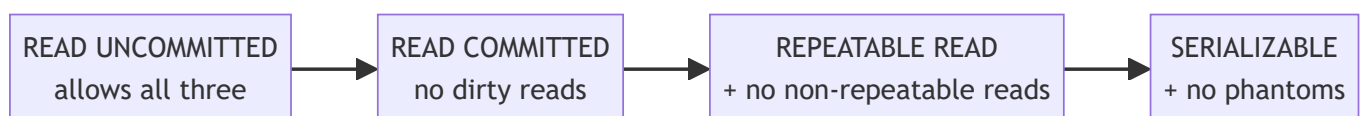
| Problem             | What changes between two reads                      | Same <b>row</b> or new <b>rows</b> ? |
|---------------------|-----------------------------------------------------|--------------------------------------|
| Lost update         | two writers overwrite each other                    | –                                    |
| Dirty read          | reading <b>uncommitted</b> data                     | a row                                |
| Non-repeatable read | a <b>committed update</b> to a row                  | the same row                         |
| Phantom read        | a committed <b>insert/delete</b> matching the query | a different set of rows              |

## E2.4 Isolation levels

The SQL standard defines four levels. A stricter level prevents more problems but reduces concurrency (more waiting or more retries).

| Isolation level  | Dirty read       | Non-repeatable read | Phantom read        |
|------------------|------------------|---------------------|---------------------|
| READ UNCOMMITTED | possible         | possible            | possible            |
| READ COMMITTED   | <b>prevented</b> | possible            | possible            |
| REPEATABLE READ  | prevented        | <b>prevented</b>    | possible (standard) |
| SERIALIZABLE     | prevented        | prevented           | <b>prevented</b>    |

*Memorise it as a staircase: each level removes one more problem.*



**SERIALIZABLE** behaves as if the transactions had run one after another in some order.

## Defaults and specifics per database

| Database       | Default         | Notes                                                                                                                         |
|----------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------|
| Oracle         | READ COMMITTED  | supports only READ COMMITTED, SERIALIZABLE and READ ONLY. <b>Never allows dirty reads</b> , so READ UNCOMMITTED doesn't exist |
| MySQL (InnoDB) | REPEATABLE READ | uses next-key locks, which also block most phantoms                                                                           |
| PostgreSQL     | READ COMMITTED  | READ UNCOMMITTED behaves as READ COMMITTED; REPEATABLE READ also prevents phantoms (snapshot isolation)                       |
| SQL Server     | READ COMMITTED  | locking by default; READ_COMMITTED_SNAPSHOT and SNAPSHOT options give row versioning                                          |

Setting the level in Oracle:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  -- must be the first statement of the transaction
SET TRANSACTION READ ONLY;                      -- sees a frozen snapshot; no DML allowed
ALTER SESSION SET ISOLATION_LEVEL = SERIALIZABLE; -- for the whole session
```

## How Oracle does it — read consistency (MVCC)

Oracle uses **multi-version concurrency control**. When you run a query, you see the data **as it was at the moment the statement started** (or, in a serializable / read-only transaction, when the *transaction* started). To rebuild an older version of a row, Oracle reads the old values from **undo**.

The result:

- **Readers never block writers, and writers never block readers.** A long report doesn't stop people updating data.
- **Writers block only other writers of the *same row*.**
- There are no dirty reads at all.

Two Oracle-specific errors follow from this design:

| Error                                                        | Meaning                                                                                                                              |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>ORA-08177</b> can't serialize access for this transaction | in SERIALIZABLE mode, another transaction changed a row you are trying to change after your transaction began. Retry the transaction |
| <b>ORA-01555</b> snapshot too old                            | a long-running query needed old undo data that has been overwritten. Fix: bigger undo retention, shorter queries                     |

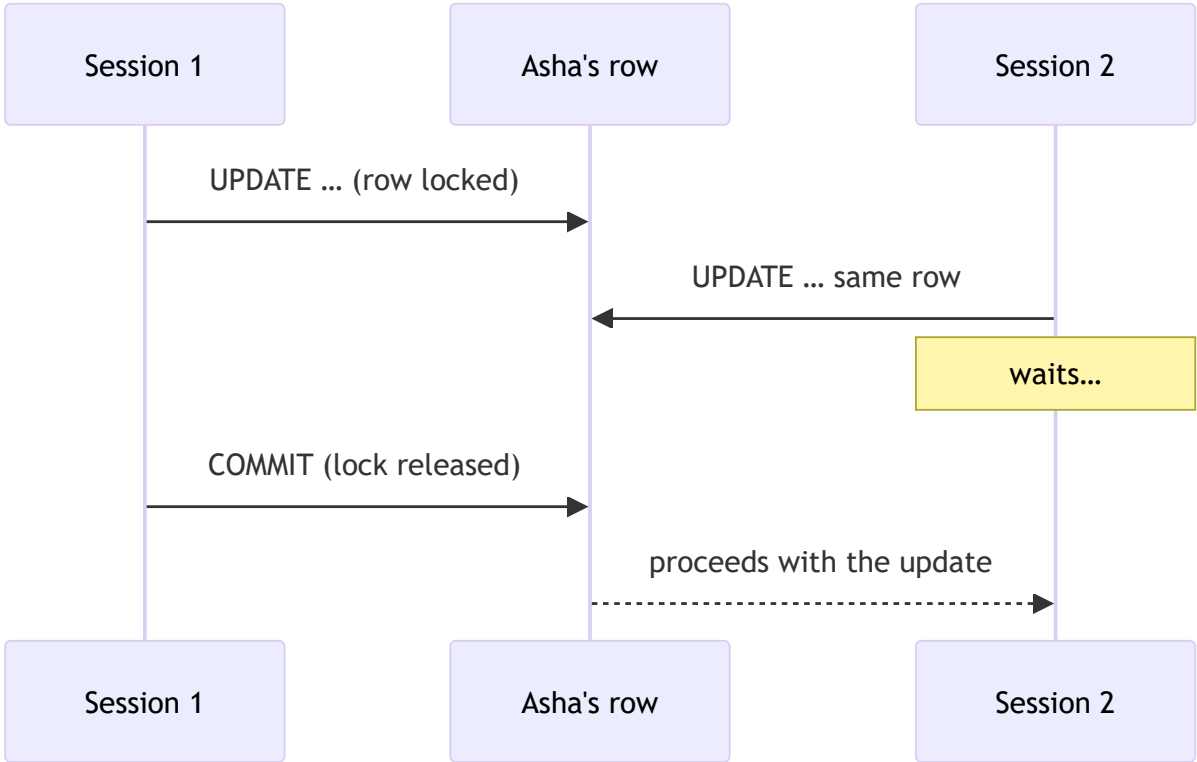
## E2.5 Locks

To keep writers from colliding, databases lock rows (or larger objects).

| Lock                          | Effect                                                                     |
|-------------------------------|----------------------------------------------------------------------------|
| <b>Shared (read)</b> lock     | many transactions may hold it; blocks writers                              |
| <b>Exclusive (write)</b> lock | one holder only; blocks other writers (and, in locking databases, readers) |
| <b>Row-level</b>              | only the touched rows (Oracle default for DML)                             |

| Lock               | Effect                                      |
|--------------------|---------------------------------------------|
| <b>Table-level</b> | whole table (DDL, <code>LOCK TABLE</code> ) |

In Oracle an `UPDATE` / `DELETE` puts a **row lock** on each row it changes; a second session trying to change the same row **waits** until the first one commits or rolls back. That waiting is called **blocking**.



### `SELECT ... FOR UPDATE` — lock rows on purpose

A normal `SELECT` takes no locks. To read a row *and reserve it* until you finish (pessimistic locking):

```

SELECT balance FROM accounts WHERE owner = 'Asha' FOR UPDATE;
-- other sessions cannot update or delete this row until you COMMIT / ROLLBACK
UPDATE accounts SET balance = balance - 300 WHERE owner = 'Asha';
COMMIT;
  
```

| Variant                             | Behaviour when the row is already locked                                                             |
|-------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>FOR UPDATE</code>             | wait indefinitely                                                                                    |
| <code>FOR UPDATE NOWAIT</code>      | fail immediately with <b>ORA-00054</b> resource busy                                                 |
| <code>FOR UPDATE WAIT 5</code>      | wait up to 5 seconds, then <b>ORA-30006</b>                                                          |
| <code>FOR UPDATE SKIP LOCKED</code> | silently <b>skip</b> locked rows — perfect for job queues where several workers grab different tasks |

This fixes the **lost update** from §E2.3: because T1 reserves the row, T2's read-for-update must wait for T1's commit and then sees 1200, not 1000.

## E2.6 Optimistic vs pessimistic concurrency

|            | Pessimistic                  | Optimistic                                      |
|------------|------------------------------|-------------------------------------------------|
| Assumption | conflicts are likely         | conflicts are rare                              |
| Technique  | lock first ( FOR UPDATE )    | don't lock; <b>check for change when saving</b> |
| Cost       | waiting, possible deadlocks  | occasional retry                                |
| Best for   | short transactions, hot rows | web apps, "user edits a form for minutes"       |

The usual optimistic technique is a **version column**: the `UPDATE` succeeds only if nobody changed the row since you read it.

```
CREATE TABLE item (id INTEGER PRIMARY KEY, stock INTEGER, version INTEGER);
INSERT INTO item VALUES (1, 10, 1);
-- Both users read: stock = 10, version = 1.
-- User A saves first: works because version is still 1.
UPDATE item SET stock = 8, version = version + 1 WHERE id = 1 AND version = 1;
-- User B saves next with the version it read (1): the row now has version 2, so 0 rows match.
UPDATE item SET stock = 7, version = version + 1 WHERE id = 1 AND version = 1;
```

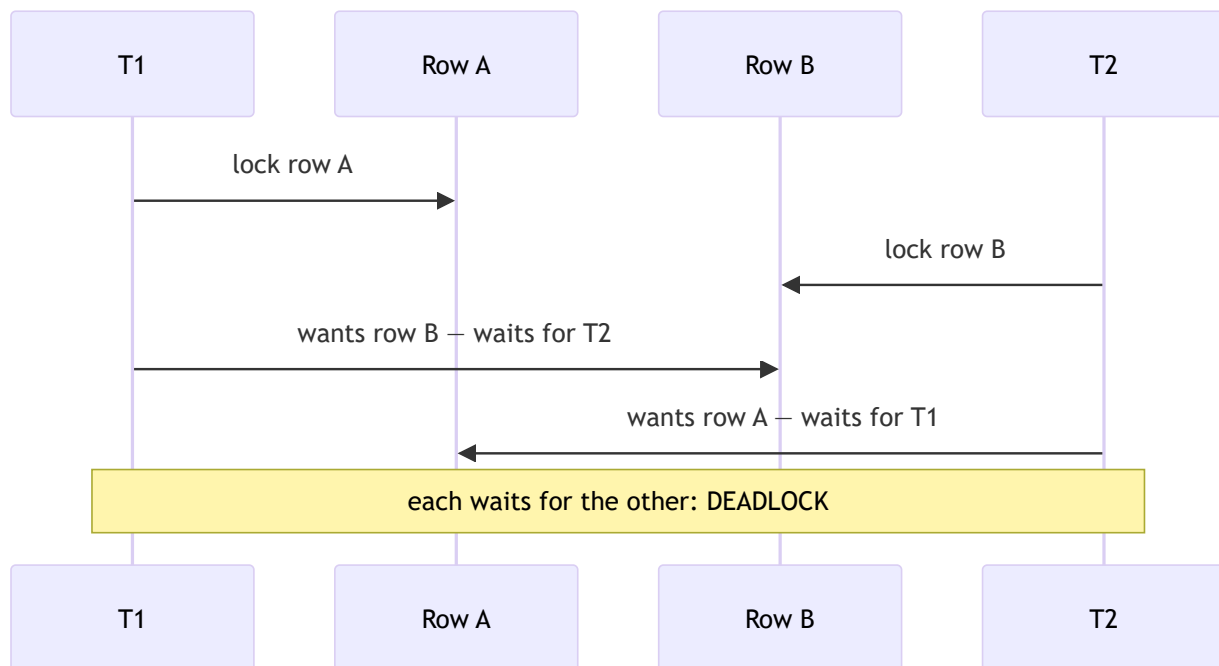
### Table after

| id | stock | version |
|----|-------|---------|
| 1  | 8     | 2       |

User B's `UPDATE` changed **0 rows** — the application sees that and tells B to reload and retry instead of silently overwriting A's change. (The table below shows the final state: stock 8, version 2.)

## E2.7 Deadlocks

A **deadlock** happens when two transactions each hold a lock the other needs, so both wait forever.



The database detects the cycle and **kills one statement** (the "victim"): Oracle raises **ORA-00060** `deadlock detected while waiting for resource` in one session, which rolls back *that statement* (not the whole transaction) and leaves the other session to continue. SQL Server reports error 1205; PostgreSQL error 40P01; MySQL error 1213.

### Preventing deadlocks

1. **Always touch rows/tables in the same order** in every transaction (the classic fix — the diagram above only happens because T1 goes A→B and T2 goes B→A).
2. Keep transactions **short**; never wait for user input inside one.
3. Lock everything you need up front ( `SELECT ... FOR UPDATE` , in a consistent order).
4. Index foreign keys — in Oracle, an unindexed foreign key can cause table-level locks and surprising deadlocks when the parent is changed.
5. Catch the deadlock error in the application and **retry**.

## E2.8 COMMIT and ROLLBACK — details worth knowing

| Point                      | Fact                                                                                                                                                                                     |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDL                        | <code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code> <b>commit implicitly</b> in Oracle (before and after)                                               |
| Ending a session           | SQL*Plus <b>commits</b> pending work on a normal <code>EXIT</code> ; an abnormal disconnect or crash <b>rolls it back</b>                                                                |
| <code>ROLLBACK</code> cost | proportional to the amount of change (undo must be applied); <code>COMMIT</code> is always fast                                                                                          |
| Statement-level atomicity  | if <b>one statement</b> fails partway (say an <code>UPDATE</code> hits a constraint on row 500), Oracle undoes <b>just that statement</b> ; earlier statements in the transaction remain |
| Autonomous transaction     | a PL/SQL block with <code>PRAGMA AUTONOMOUS_TRANSACTION</code> runs its own independent transaction — commonly used for audit logging that must survive a rollback                       |
| Distributed transaction    | spans several databases; needs two-phase commit (2PC)                                                                                                                                    |

**Two-phase commit in one line:** a coordinator asks every participant "*can you commit?*" (prepare); only if **all** say yes does it tell them to *commit* — otherwise everyone rolls back.

---

## Interview corner

---

**Q. What is a transaction? What is ACID?** A logical unit of work executed entirely or not at all. Atomicity — all or nothing; Consistency — moves the database between valid states; Isolation — concurrent transactions don't interfere; Durability — committed data survives failures.

**Q. Explain dirty read, non-repeatable read and phantom read.** Dirty: reading another transaction's *uncommitted* change. Non-repeatable: re-reading the *same row* gives a different value because another transaction committed an update. Phantom: re-running the *same query* returns different *rows* because another transaction committed an insert/delete.

**Q. List the isolation levels and what each prevents.** Read Uncommitted (nothing), Read Committed (dirty reads), Repeatable Read (+ non-repeatable reads), Serializable (+ phantoms).

**Q. What is Oracle's default level, and does it allow dirty reads?** `READ COMMITTED`, and Oracle never allows dirty reads — it gives each statement a consistent snapshot using undo (MVCC), so readers don't block writers.

**Q. What is a deadlock and how do you avoid it?** Two transactions each waiting for a lock the other holds. The database picks a victim (ORA-00060). Avoid by acquiring locks in a consistent order, keeping transactions short, and retrying.

**Q. Optimistic vs pessimistic locking?** Pessimistic locks rows up front ( `SELECT ... FOR UPDATE` ). Optimistic doesn't lock; it checks a version/timestamp at update time and retries on conflict.

**Q. What does `SKIP LOCKED` do?** Ignores rows that other sessions have locked, so parallel workers can each take different rows from a queue table.

**Q. Does `TRUNCATE` or `DROP` roll back?** No in Oracle/MySQL (DDL commits implicitly). PostgreSQL and SQL Server allow DDL inside a transaction and roll it back.

**Q. What is the difference between `ROLLBACK` and `ROLLBACK TO SAVEPOINT` ?** `ROLLBACK` ends the transaction and undoes everything; `ROLLBACK TO` undoes only back to the savepoint and the transaction continues.

---

## Quick recap

---

- A transaction = all-or-nothing unit of work; Oracle starts one at your first DML, ends on `COMMIT` / `ROLLBACK` /DDL.
- **ACID** = Atomicity · Consistency · Isolation · Durability (undo makes rollback possible; redo makes commits durable).
- Anomalies: **lost update, dirty read, non-repeatable read, phantom.**
- Isolation levels: Read Uncommitted → Read Committed → Repeatable Read → Serializable; **Oracle: Read Committed (default), Serializable, Read Only — no dirty reads.**
- Oracle reads never block writes (MVCC via undo); `SELECT ... FOR UPDATE [NOWAIT | WAIT n | SKIP LOCKED]` locks rows deliberately.
- **Deadlock** = circular wait → ORA-00060; prevent with consistent lock order and short transactions.
- Optimistic = version column, check at update; pessimistic = lock first.